Test Automation Workflow

A practical reference for designing maintainable UI tests

Recommended order

1. **Understand the business scenario**
   Write down the user journey and expected outcome before choosing implementation details.
2. **Identify pages and components**
   Map each step to a page object or reusable UI component. Use a component for a shared part of a page; use a page object for a full page.
3. **Identify the required elements**
   List the fields, buttons, links, headings, and result areas the flow will use or verify.
4. **Define the locators**
   Create or reuse locators in the appropriate locator class. Keep locator definitions focused on identifying elements.
5. **Implement page objects and components**
   Have each object create/use its locators, expose useful elements, and provide methods for page-specific actions. Return the next page object when an action navigates.
6. **Prepare data and fixtures when needed**
   Use fixtures for shared, generated, or set-up data. Use the data reader for external Excel/JSON data. Keep simple test-specific values in the test when appropriate.
7. **Write the test flow**
   Call page-object/component methods in business order, then assert expected behavior with Playwright expect. Keep low-level locator details out of the test when practical.
8. **Add reusable utilities only when appropriate**
   Put genuinely shared, non-page-specific behavior in utils/. Keep page-specific actions in their page object or component.
9. **Run and refine**
   Execute the test and verify locators, navigation, data, and assertions together. Keep the test focused on the intended behavior.

Short version

Business scenario -> pages/components -> required elements -> locators ->
page-object actions -> data/fixtures as needed -> test actions and assertions -> validation

Use existing project patterns and code; a test does not need a new class, fixture, or utility for every step.